

Programlama Dilleri ve Yazılım Geliştirme

Temel Kodlama Mantığı, Algoritmalar, Derleyici Mimarileri, SDLC Yaşam Döngüsü ve Modern Yazılım Paradigmaları

KBU Ders Sunumu | Öğr. Gör. Saliha Gülsen KESKİN

BÖLÜM 1

Programlama Kavramı ve Dil Mimarisi

Programlama dili nedir, insan dilinden makine diline komut aktarımı ve derleyici mekanizmaları.

Programlama Dili Nedir? (Mutfak Tarifi)

Akademik Tanım ve Köprü İşlevi

Programlama dili, geliştiricilerin belirli görevleri bilgisayara iletmesi için tasarlanmış kurallar dizisidir:

- İnsanların okuyup anlayabileceği sözdizimsel (Syntax) yapılara sahiptir.
- Bilgisayara ne yapması gerektiğini hassas adımlarla söyler.

Yemek Tarifi Benzetmesi

Kodlama mantığı aslında mutfaktaki tarif defterine benzer:

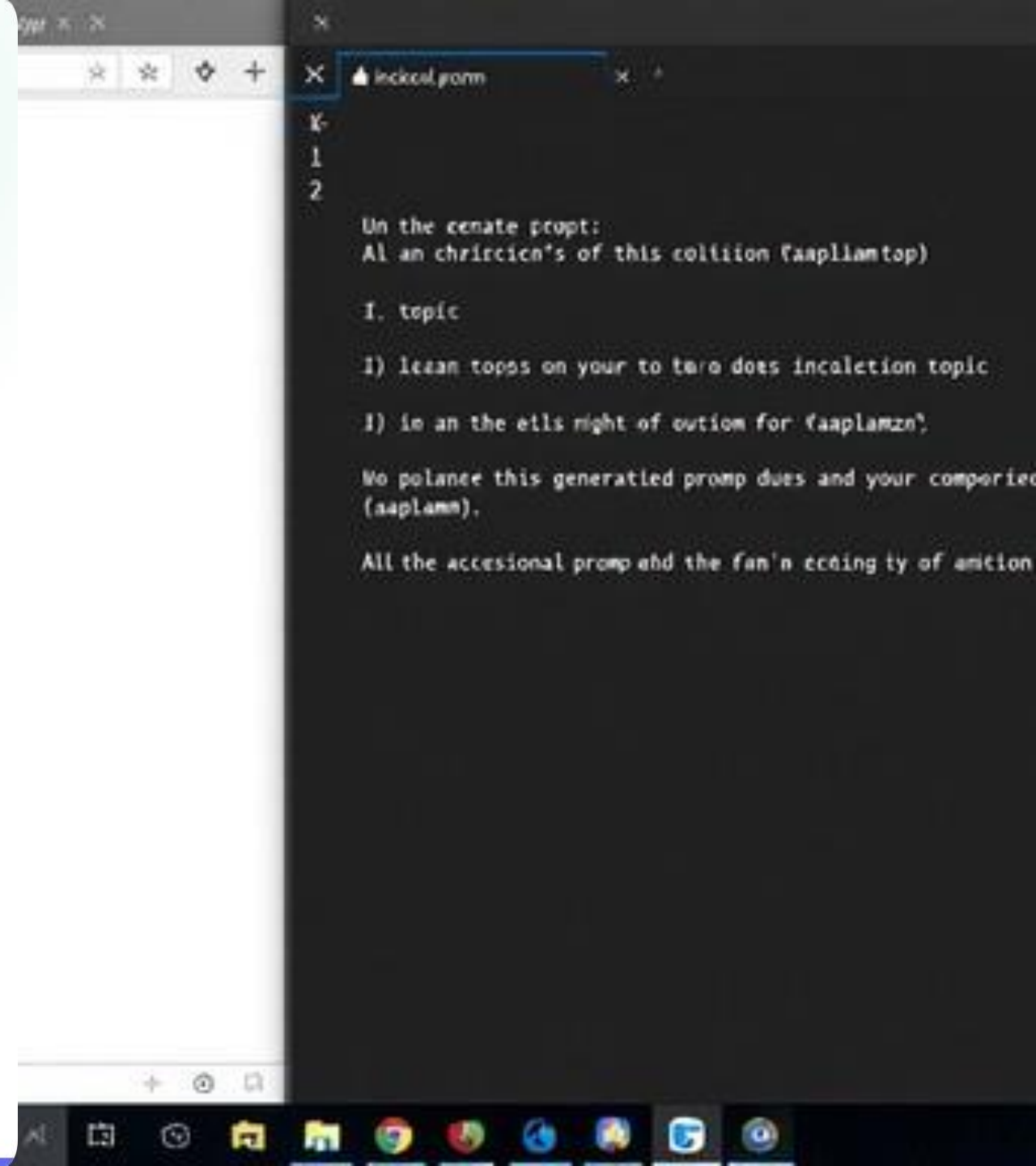
- **Aşçı (İşlemci / CPU):** Tarifi harfiyen uygulayan birim.
- **Yemek Tarifi (Program):** Sırayla yapılması gereken komut adımları.
- **Malzemeler (Veri):** İşlenecek ham girdiler.

Kaynak Kod (Source Code) vs. Makine Dili

İnsan Kodundan İkili Mantığa

Kaynak Kod: Yazılımcının yüksek seviyeli dilde (C++, Java, Python) yazdığı okunabilir metin belgesidir.

Makine Dili (Binary): İşlemcinin doğrudan anladığı 0 ve 1 sinyallerinden oluşan donanım seviyesindeki dizilimdir.



Derleyici (Compiler) ve Yorumlayıcı

Derleyici (Compiler) Mantığı

Tüm kaynak kodu tek seferde makine koduna çeviren çevirici programdır:

- Tüm dosyayı tarar, sentaks (yazım) hatalarını bulur ve `.exe` üretir.
- Çalışma hızı çok yüksektir. Örn: C, C++, Rust, Go.

Yorumlayıcı (Interpreter) Mantığı

Kaynak kodu satır satır okuyup anında icra eden mekanizmadır:

- Hata ile karşılaştığı ilk satırda çalışmayı durdurur.
- Hızlı prototipleme ve test imkânı sunar. Örn: Python, JavaScript, PHP.

Çalışma Türlerine Göre Diller

Çalışma Türü	Çeviri Mekanizması	Avantajları	Örnek Diller
Derlenen (Compiled)	Tüm kod önceden makine koduna çevilir.	Maksimum çalışma hızı ve donanım performansı	C, C++, Rust, Go
Yorumlanan (Interpreted)	Kod satır satır canlı yürütülür.	Kolay hata ayıklama ve platform bağımsızlık	Python, JavaScript, Ruby, PHP
Hibrit (Bytecode)	Önce Bayt Koda derlenir, sanal makinede (JVM) yorumlanır.	"Bir kez yaz, her yerde çalıştır" esnekliği	Java, C#, Kotlin

Programlama Dillerinin Tarihsel Evrimi

1940 - 1970 Dönemi

Makine dilinden ilk soyutlanmış dillere geçiş dönemi:

- **1940-50:** Delikli kartlar, Makine dili ve Assembly.
- **1957 (Fortran) & 1959 (COBOL):** İlk yüksek seviyeli bilimsel ve ticari diller.
- **1972 (C Dili):** Modern işletim sistemlerinin (Unix) temeli.

1980 - Günümüz

Nesne yönelimli ve modern dinamik dillerin doğuşu:

- **1980-90 (C++, Java, C#):** Nesne Yönelimli Programlama (OOP) devrimi.
- **2000+:** Python, JavaScript, Swift, Go ve Rust ile yüksek verimlilik.

BÖLÜM 2

Programlama Dillerinin Sınıflandırılması

Seviyelerine göre dil hiyerarşisi, uygulama alanları ve ekosistem haritası.

Seviyelerine Göre Programlama Dilleri

Düşük ve Orta Seviyeli Diller

Donanıma ve makine mimarisine yakın dillerdir:

- **Düşük Seviye (Assembly):** Doğrudan işlemci yazmaçlarına erişir; yazılması zor fakat aşırı hızlıdır.
- **Orta Seviye (C, C++):** Hem belleği doğrudan yönetir hem de modüler yapı sunar.

Yüksek ve Çok Yüksek Seviyeli Diller

İnsan diline ve mantığına yakın soyutlanmış dillerdir:

- **Yüksek Seviye (Java, C#, Python):** Otomatik bellek yönetimi (Garbage Collector) barındırır.
- **Çok Yüksek Seviye (SQL, Visual Basic):** Görsel arayüzler ve sorgu dili odaklıdır.

Kullanım Alanlarına Göre Programlama Dilleri

Uygulama Alanı	Frontend / İstemci	Backend / Sunucu	Öne Çıkan Diller
Web Geliştirme	HTML, CSS, JavaScript, TypeScript	Node.js, Python, C#, PHP	JS, TS, Python, C#
Mobil Uygulama	Swift (iOS), Kotlin (Android)	REST API Services	Flutter (Dart), React Native
Oyun Geliştirme	Unity (C#), Unreal Engine (C++)	Game Server Infrastructure	C++, C#
Yapay Zeka & Veri Bilimi	Jupyter, Streamlit	TensorFlow, PyTorch Core	Python, R, Julia

En Popüler Modern Programlama Dilleri



Python

Sade sözdizimi ile Yapay Zeka, Veri Bilimi ve otomasyon alanının lideri.



JavaScript

Web tarayıcılarının ve modern full-stack web uygulamalarının kalbi.



Java & C#

Kurumsal şirket sistemleri, bankacılık ve Android/Unity ekosistemleri.



C++ & Rust

Sistem programlama, oyun motorları ve bellek güvenliği odaklı diller.

BÖLÜM 3

Algoritmik Düşünme ve Akış Diyagramları

Problem çözme sanatı, mantıksal adım tasarımı ve görsel akış şemaları.

Algoritma Nedir? Problem Çözme Mantığı

Adım Adım Çözüm Yolu

Algoritma: Belli bir problemi çözmek veya bir hedefe ulaşmak için tasarlanan sonlu mantıksal adımlar kümesidir.

İsmi Harezmi'den (Al-Khwarizmi) alır. Bir başlangıç noktasıyla başlar ve net bir çıktıyla sonlanır.

t/stop

r action

dition

link



Bir Algoritmanın 5 Temel Özelliği



1. Girdi (Input)

Sisteme dışarıdan sağlanan sıfır veya daha fazla veri elemanı.



2. Kesinlik

Her komut adımının açık, net ve belirsizlikten uzak olması.



3. Sonluluk

Algoritmanın sonsuz döngüye girmeden durması.



4. Çıktı (Output)

İşlem sonunda elde edilen en az bir anlamlı sonuç.

Akış Diyagramı (Flowchart) Sembolleri

Geometrik Şekil	Sembol Adı	Temsil Ettiği Görev / Adım
Elips / Stadyum	Başla / Bitir	Algoritmanın başlangıç ve bitiş noktalarını gösterir.
Paralelkenar	Girdi / Çıktı (I/O)	Kullanıcıdan veri alma veya ekrana sonuç yazdırımı.
Dikdörtgen	İşlem (Process)	Matematiksel hesaplamalar ve değişkene değer atama.
Eşkenar Dörtgen (Baklava)	Karar / Koşul (Decision)	Mantıksal karşılaştırmalar (Evet/Hayır kontrolleri).

Örnek Algoritma: Faktöriyel Hesaplama

Matematiksel İfade ve Mantık

Bir n sayısının faktöriyeli ($n!$), 1 'den n 'e kadar olan sayıların çarpımıdır:

$$n! = 1 \times 2 \times \dots \times n$$

Adım Adım Algoritma Metni

- Adım 1: Başla.
- Adım 2: Kullanıcıdan n sayısını al.
- Adım 3: `faktöriyel = 1`
- Adım 4: $n > 0$ olduğu sürece `faktöriyel = faktöriyel * n`
- Adım 5: `faktöriyel` na yazdır ve Bitir.

BÖLÜM 4

Temel Programlama Yapıları

Değişkenler, veri tipleri, karar mekanizmaları, döngüler ve fonksiyonlar.

Değişkenler (Variables) ve Kutu Mantığı

Değişken Nedir? (Etiketli Kutu)

Değişken, bilgisayarın geçici belleğinde (RAM) bir değeri saklamak için ayrılan isimli alandır:

- Etiketli bir kutu gibidir. İçine veri konur ve değiştirilebilir.
- Değişken ismi ile bellekteki o adrese erişilir.

Atama ve Değer Değişimi

Kod içinde değer ataması eşittir (=) operatörü ile yapılır:

```
int sayi = 10; // Kutuya 10 koyduk  
sayi = 25; // Eski değer silindi, yeni değer 25
```

Temel Veri Tipleri (Data Types)

Veri Tipi	Açıklama / Temsil Ettiği Veri	Bellek Boyutu	Örnek Değer
int (Integer)	Tam sayılar (pozitif / negatif)	4 Bayt (32-bit)	4425-1500
float / double	Ondalık / rasyonel sayılar	4 / 8 Bayt	33.14159
char	Tek bir karakter / sembol	2 Bayt (Unicode)	1/2'A'%
string	Metin karakter dizileri	Dinamik	"Merhaba Dünya"
bool (Boolean)	Mantıksal doğruluk durumu	1 Bayt	truefalse

Karar ve Kontrol Yapıları (If - Else)

Mantıksal Karar Mekanizması

Programın belirlenen koşula göre farklı yolları izlemesini sağlar:

- Koşul doğru (`true`) blok çalışır.
- Koşul yanlış (`false`) öğü (`else`)

Örnek C# / Java Kod Bloğu

```
if (sifre == 1024) {  
    Console.WriteLine ("Giriş Başarılı");  
} else {  
    Console.WriteLine ("Hatalı Şifre!");  
}
```

Döngü Yapıları (For ve While)

Döngü (Loop) Nedir?

Bir kod bloğunu belirli bir koşul sağlandığı sürece defalarca çalıştırmaya yarar:

- 100 kişiye mail gönderirken 100 kez aynı kodu yazmak yerine döngü kurulur.
- **For Döngüsü:** Adım sayısı önceden belli ise tercih edilir.

For Döngüsü Örneği

```
// 1'den 5'e kadar ekrana yazdırır
for (int i = 1; i ≤ 5; i++) {
    Console.WriteLine("Sayı: " + i);
}
```

Fonksiyonlar (Metodlar) ve Modülerlik

Modüler Programlama

Büyük programları küçük, yönetilebilir parçalara bölme tekniğidir:

- Kod tekrarını önler (DRY - Don't Repeat Yourself).
- Bir kez yazılır, ihtiyaç duyulan her yerden çağrılır.

Fonksiyon Anatomisi

```
int Topla (int sayi1, int sayi2) {  
    return sayi1 + sayi2;  
}
```

// Çağırılması:

```
int sonuc = Topla (15, 25); // 40
```

Veri Yapılarına Giriş: Diziler (Arrays)

[0]

Dizilerin İlk Eleman İndeksi

Yan Yana Bellek Hücreleri

Dizi (Array): Aynı türdeki çok sayıda veriyi tek bir değişken altında sırayla saklayan veri yapısıdır.

```
int [] sayilar = {10, 20, 30, 40};  
Console.WriteLine (sayilar[0]); // Ekrana 10 yazar
```

BÖLÜM 5

Yazılım Geliştirme Yaşam Döngüsü (SDLC)

Mühendislik disiplini, gereksinim analizinden canlıya alım ve bakıma uzanan süreçler.

Yazılım Geliştirme Aşamaları (SDLC)

SistematiK Mühendislik Süreci

Bir yazılım sadece kod yazmaktan ibaret değildir. Başarılı projeler 6 ana aşamadan geçer:

1. Analiz → 2. Tasarım → 3. Kodlama → 4. Test → 5. Dağıtım → 6. Bakım

Software Development Project Team Planning

software development team resource planning and work scheduling sheet

				
	Priority	Owner	Start Date	End Date
Requirement mapping				
Requirements	High	Amit Malik	30-Nov-2021	05-Dec-2021
Users	Medium	Helmut Drabe	30-Nov-2021	04-Dec-2021
Test	Low	Shirin Sandra	04-Dec-2021	08-Dec-2021
Designing				
Features	High	Franny Sheldon	5-Dec-2021	12-Dec-2021
Resources	Low	Leonard Beaty	30-Nov-2021	01-Dec-2021
Scripts	Medium	Leonard Beaty	30-Nov-2021	01-Dec-2021
Dashboards and Charts	High	Don A, Chaney	11-Dec-2021	14-Dec-2021
Intelligence) features	Medium	Leonard Beaty	7-Dec-2021	11-Dec-2021
Writing				
Interface	High	Peggy Walters	28-Nov-2021	30-Nov-2021
Reports, dashboards and	Medium	Peggy Walters	30-Nov-2021	01-Dec-2021
Features	High	Anand Das	5-Dec-2021	11-Dec-2021

Yazılım Metodolojileri: Waterfall vs. Agile

Şelale (Waterfall) Modeli

Geleneksel ve katı doğrusal ilerleme modelidir:

- Bir aşama bitmeden diğerine geçilemez.
- Gereksinimlerin baştan çok net olduğu projelerde uygundur.

Çevik (Agile / Scrum) Metodolojisi

Modern, esnek ve yinelemeli (iteratif) süreçtir:

- Proje 2-4 haftalık kısa döngülere (Sprint) bölünür.
- Müşteri geri bildirimlerine hızlı adapte olunur.

Yazılım Geliştirici Araçları (Dev Tools)



IDE Ekosistemleri

Visual Studio, VS Code, IntelliJ IDEA gibi gelişmiş kod editörleri.



Sürüm Kontrolü (Git)

Kod geçmişini saklayan ve takımların aynı projede çalışmasını sağlayan sistem.



Test Araçları

JUnit, Postman ve Selenium ile otomatik birim ve entegrasyon testleri.



DevOps & CI/CD

Docker, Jenkins ve GitHub Actions ile sürekli entegrasyon ve dağıtım.

Geleceğin Programlama Paradigmaları

AI Destekli Kodlama ve Kuantum

Yapay Zeka Destekli Geliştirme: Copilot ve LLM'ler geliştirici verimliliğini katlamaktadır.

Kuantum Programlama: Qiskit ve Q# ile klasik bilgisayarların ötesinde algoritma tasarımı.



Yazılım Mühendisliğinde Temel İlkeler

Temel Kodlama İlkeleri

Temiz ve sürdürülebilir kod yazmanın prensipleri:

- **DRY (Don't Repeat Yourself):** Kendini tekrar etme.
- **KISS (Keep It Simple, Stupid):** Mümkün olduğunca basit tut.

Kariyer ve Uzmanlaşma Yolları

Geleceğin yazılımcıları için odak alanları:

- Full-Stack Web Geliştirme, Mobil Yazılım.
- Yapay Zeka / Makine Öğrenmesi Mühendisliği.
- Siber Güvenlik ve Bulut Mimarisi (Cloud Computing).

Sorular ve Tartışma

Programlama Dilleri ve Yazılım Geliştirme dersimizin sonuna geldik.

Image Sources



https://images.stockcake.com/public/f/4/9/f495ff07-569e-404c-a1ff-5d5bb131fe8a_large/dark-code-editor-stockcake.jpg

Source: stockcake.com



https://www.digitaltechnologieshub.edu.au/media/gc5c5ni1/have-fun-with-flowcharts_thumbnailpng.png?width=480&height=301&format=webp&quality=100

Source: www.digitaltechnologieshub.edu.au



https://www.slideteam.net/media/catalog/product/cache/1280x720/a/g/agile_software_development_project_team_planning_agile_project_team_planning_it_slide01.jpg

Source: www.slideteam.net



<https://www.itpt.co.uk/wp-content/uploads/2022/07/data-science-python-programming-AI-and-ML.png>

Source: www.itpt.co.uk